CI/CD Overview

CI (Continuous Integration): The practice of automatically integrating code into a shared repository multiple times a day. It helps detect issues early by frequently testing the software.

CD (Continuous Delivery/Continuous Deployment):

- **Continuous Delivery**: The practice of automatically deploying code to production-like environments after passing all tests.
- **Continuous Deployment**: Extends Continuous Delivery by automatically deploying to production once code passes the tests.

GitLab CI/CD

Introduction to GitLab CI/CD

GitLab CI/CD allows teams to build, test, and deploy code automatically through pipelines. These pipelines are defined in `.gitlab-ci.yml` files in your repository.

PATHNEX
Shaping Careers, Building Futures

Key Components in GitLab CI/CD

1. **Pipeline**: A sequence of stages executed in order. It defines the steps needed for building, testing, and deploying the code.
2. **Jobs**: Tasks that are run as part of the pipeline, such as compiling code, running tests, etc.
3. **Stages**: Logical grouping of jobs in a pipeline (e.g., `build`, `test`, `deploy`).
4. **Runners**: Agents that run the jobs. GitLab can use shared runners or custom runners configured by the user.

Basic GitLab CI/CD Configuration

A typical `.gitlab-ci.yml` file consists of jobs and stages.

Example:

```
yaml

stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Building project..."

test_job:
  stage: test
```

```
  script:
    - echo "Running tests..."

deploy_job:
  stage: deploy
  script:
    - echo "Deploying to production..."
```

Pipeline Execution

1. GitLab automatically triggers the pipeline when you push changes to the repository.
2. The pipeline runs in the order of the defined stages.
3. Each job in a stage can depend on the output of the previous job, and if a job fails, the pipeline halts.

Jenkins CI/CD

Introduction to Jenkins CI/CD

Jenkins is an open-source automation server used for continuous integration and continuous delivery. It supports integration with many version control systems (e.g., GitHub, Bitbucket) and allows automated testing and deployment.

PATHNEX
Shaping Careers, Building Futures

Key Components in Jenkins CI/CD

1. **Pipeline**: The series of steps (jobs) that Jenkins will execute. These can be created using the Jenkins UI or by using a `Jenkinsfile`.
2. **Jenkinsfile**: A file that defines the pipeline script. It uses **Declarative Pipelines** or **Scripted Pipelines** to define the CI/CD process.
3. **Jobs**: Tasks within a pipeline, such as compiling code, running tests, and deploying the application.
4. **Nodes and Executors**: Jenkins runs jobs on specific machines known as nodes. Executors are the processes that run on these nodes.
5. **Freestyle Projects**: A simpler configuration for jobs but less flexible than Pipelines.

Jenkins Pipeline Example (Declarative)

```
groovy

pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
                // commands to build
            }
        }

        stage('Test') {
```

```
            steps {
                echo 'Running tests...'
                // commands to test
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying to production...'
                // commands to deploy
            }
        }
    }
}
```

Pipeline Execution

1. **Trigger**: Jenkins pipelines can be triggered by events such as Git commits, scheduled times, or manual triggers.
2. **Stages**: Each stage runs sequentially unless parallel execution is defined.
3. **Post Actions**: Post-execution actions (such as sending notifications) can be defined in the pipeline.

Common Concepts in CI/CD (GitLab(Repo) + Jenkins(CI/CD))

PATHNEX
Shaping Careers, Building Futures

1. **Version Control Integration**: Both GitLab and Jenkins integrate with Git (and other version control systems) to trigger pipelines based on code changes.
2. **Automated Testing**: Both platforms support running automated tests as part of the pipeline to catch bugs early in the development process.
3. **Environment Configuration**: Both support deployment to various environments (e.g., staging, production) after successful tests.
4. **Artifact Management**: You can store built artifacts (e.g., `.jar` or `.zip` files) and deploy them to different environments.

Best Practices for CI/CD

1. **Use Branching Strategies**: Feature branches, release branches, and main branches should all trigger different workflows.
2. **Fast Feedback**: Keep feedback loops fast. Break down your pipeline into smaller, faster jobs (e.g., unit tests first).
3. **Environment Parity**: Make sure the environments used for testing and production are similar to avoid "works on my machine" issues.
4. **Keep Pipelines Efficient**: Avoid unnecessary tasks. Run critical tasks first and avoid duplicating work.
5. **Version Control Everything**: Ensure that your pipeline configurations (`.gitlab-ci.yml`, `Jenkinsfile`) and deployment scripts are version-controlled.

Integrating GitLab with Jenkins (Hybrid CI/CD)

If you want to integrate GitLab and Jenkins for CI/CD, you can trigger Jenkins jobs from GitLab using **GitLab webhooks**.

Steps to Trigger Jenkins from GitLab

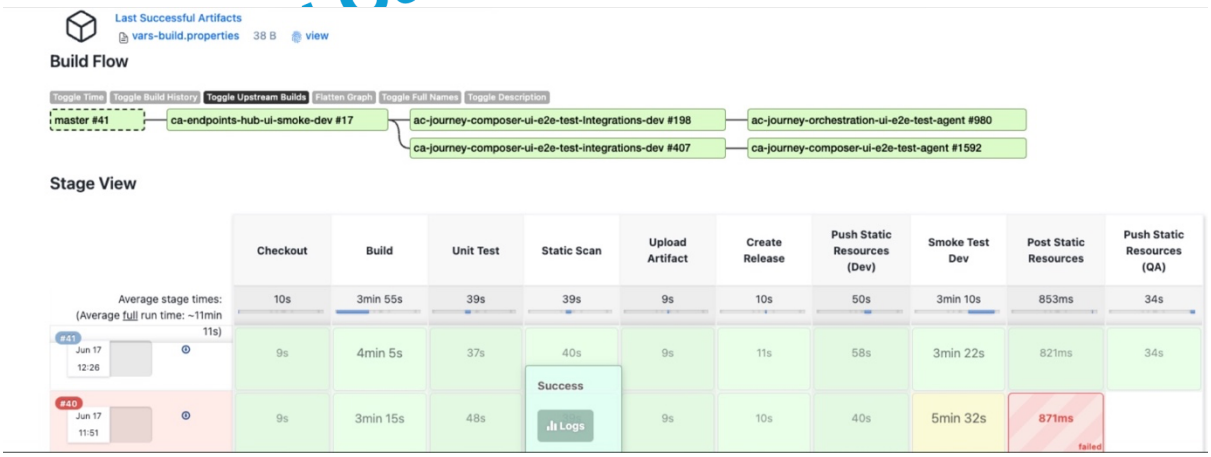
1. In GitLab, set up a **webhook** to trigger a Jenkins job.
2. In Jenkins, configure the job to listen for GitLab push events.
3. When a developer pushes changes to GitLab, the webhook will notify Jenkins, and Jenkins will start the pipeline for that commit.

Useful Plugins for Jenkins

- **GitLab Plugin**: Allows Jenkins to receive GitLab webhook events.
- **Docker Pipeline**: For integrating Docker into Jenkins pipelines.
- **JUnit Plugin**: For reporting test results.
- **Slack Notification Plugin**: For sending notifications to Slack after pipeline completion.

PATHNEX
Shaping Careers, Building Futures

Real Pipeline (Important for stages)



Real Scenario Jenkins CI/CD Pipeline

```
pipeline {
  agent any

  environment {
    NODE_ENV = 'production'
    DEPLOY_SERVER = 'your.server.com'
    DEPLOY_PATH = '/var/www/your-app'
    SSH_PRIVATE_KEY = credentials('ci-secret-ssh-private-key') // Assuming you have
stored the private key as a Jenkins credential
    DOCKER_IMAGE = 'your-docker-image'
    TAG_VERSION = "v${env.BRANCH_NAME}"
  }

  stages {
    // Stage: Lint the Code
    stage('Lint Code') {
      steps {
        script {
          sh 'npm install -g eslint'
          sh 'eslint . --ext .js,.jsx,.ts,.tsx'
        }
      }
      when {
        branch 'master'
        branch 'PR-*'
      }
      tags 'node'
    }
    // Stage: Install Dependencies
    stage('Install Dependencies') {
      steps {
        script {
          sh 'npm install'
        }
      }
      post {
        always {
          cache(path: 'node_modules', key: 'node-modules')
        }
      }
      when {
        branch 'master'
      }
      tags 'node'
```

PATHNEX
Shaping Careers, Building Futures

```
}

// Stage: Run Tests
stage('Run Tests') {
  steps {
    script {
      sh 'npm test'
    }
  }
  when {
    branch 'master'
    branch 'PR-*'
  }
  tags 'node'
}

// Stage: Build the Application
stage('Build Application') {
  steps {
    script {
      sh 'npm run build'
    }
  }
  post {
    success {
      archiveArtifacts artifacts: 'dist/**/*', allowEmptyArchive: true
    }
  }
  when {
    branch 'master'
  }
  tags 'node'
}

// Stage: Tag Version (Tagging the commit)
stage('Tag Version') {
  steps {
    script {
      if (env.BRANCH_NAME == 'master') {
        def tag = "v${new Date().format('yyyyMMddHHmmss')}"
        sh "git tag ${tag}"
        sh "git push origin ${tag}"
      }
    }
  }
  when {
    branch 'master'
```

PATHNEX
Shaping Careers, Building Futures

```
        }
        tags 'node'
    }

    // Stage: Deploy to Staging
    stage('Deploy to Staging') {
        steps {
            script {
                sh """
                    echo "Deploying to staging..."
                    ssh -i ${SSH_PRIVATE_KEY} user@${DEPLOY_SERVER} "cd ${DEPLOY_PATH}
&& git pull origin master && npm install && npm run start"
                """
            }
        }
        when {
            branch 'master'
        }
        input {
            message 'Deploy to Staging'
            ok 'Deploy'
        }
        tags 'deploy'
    }

    // Stage: Deploy to Production (requires manual approval)
    stage('Deploy to Production') {
        steps {
            script {
                sh """
                    echo "Deploying to production..."
                    ssh -i ${SSH_PRIVATE_KEY} user@${DEPLOY_SERVER} "cd ${DEPLOY_PATH}
&& git pull origin master && npm install && npm run start"
                """
            }
        }
        when {
            tag 'v*'
        }
        input {
            message 'Deploy to Production'
            ok 'Deploy'
        }
        tags 'deploy'
    }

    // Stage: Cleanup Build Artifacts (optional)
```

PATHNEX
Shaping Careers, Building Futures

```
    stage('Cleanup Build Artifacts') {
      steps {
        script {
          sh 'rm -rf dist/'
        }
      }
      when {
        branch 'master'
      }
      tags 'node'
    }
  }

  post {
    always {
      echo "Pipeline completed"
    }
  }
}
```

PATHNEX
Shaping Careers, Building Futures

Real Scenario Gitlab Pipeline (`.gitlab-ci.yml`)

```
stages:
  - lint
  - install
  - test
  - build
  - tag
  - deploy_staging
  - deploy_production
  - cleanup

variables:
  NODE_ENV: production
  DEPLOY_SERVER: your.server.com
  DEPLOY_PATH: /var/www/your-app
  SSH_PRIVATE_KEY: $CI_SECRET_SSH_PRIVATE_KEY  # This should be configured as a CI/CD variable in GitLab
  DOCKER_IMAGE: your-docker-image
  TAG_VERSION: "v${CI_COMMIT_REF_NAME}"
```

```
before_script:
  - apt-get update -y
  - apt-get install -y ssh
  - echo "$SSH_PRIVATE_KEY" > /tmp/id_rsa
  - chmod 600 /tmp/id_rsa
  - export SSH_KEY_PATH=/tmp/id_rsa

# Stage: Lint the Code
lint_code:
  stage: lint
  script:
    - npm install -g eslint
    - eslint . --ext .js,.jsx,.ts,.tsx
  only:
    - merge_requests
    - master
  tags:
    - node

# Stage: Install Dependencies
install_dependencies:
  stage: install
  script:
    - npm install
  cache:
    paths:
      - node_modules/
  only:
    - master
  tags:
    - node

# Stage: Run Tests
run_tests:
  stage: test
  script:
    - npm test
  only:
    - master
    - merge_requests
  tags:
    - node

# Stage: Build the Application
build_application:
  stage: build
  script:
```

PATHNEX
Shaping Careers, Building Futures

```
  - npm run build
 artifacts:
  paths:
   - dist/
  expire_in: 1 hour
 only:
  - master
 tags:
  - node

# Stage: Tag Version (Tagging the commit)
tag_version:
 stage: tag
 script:
  - |
   if [ "$CI_COMMIT_REF_NAME" == "master" ]; then
    TAG="v$(date +'%Y%m%d%H%M%S')"
    git tag $TAG
    git push origin $TAG
   fi
 only:
  - master
 tags:
  - node

# Stage: Deploy to Staging
deploy_staging:
 stage: deploy_staging
 script:
  - echo "Deploying to staging..."
  - ssh -i $SSH_PRIVATE_KEY user@$DEPLOY_SERVER "cd $DEPLOY_PATH && git pull
origin master && npm install && npm run start"
 only:
  - master
 when: manual
 tags:
  - deploy

# Stage: Deploy to Production (requires manual approval)
deploy_production:
 stage: deploy_production
 script:
  - echo "Deploying to production..."
  - ssh -i $SSH_PRIVATE_KEY user@$DEPLOY_SERVER "cd $DEPLOY_PATH && git pull
origin master && npm install && npm run start"
 only:
  - tags
```

PATHNEX
Shaping Careers, Building Futures

```
  when: manual
  tags:
    - deploy

# Stage: Cleanup Build Artifacts (optional)
cleanup_artifacts:
  stage: cleanup
  script:
    - echo "Cleaning up build artifacts..."
    - rm -rf dist/
  only:
    - master
  when: always
  tags:
    - node
```

PATHNEX
Shaping Careers, Building Futures

Real Scenario CI/CD code for Terraform

```
stages:
  - install
  - lint
  - validate
  - plan
  - apply
  - cleanup

variables:
  TERRAFORM_VERSION: "1.5.0"  # Specify your desired Terraform version
  TF_STATE_BUCKET: "your-terraform-state-bucket"  # Replace with your state bucket (e.g., S3, GCS)
  TF_STATE_KEY: "terraform/state.tfstate"  # Path to your state file in the bucket
  TF_VAR_region: "us-east-1"  # Example AWS region, adjust to your cloud provider and region
  TF_VAR_environment: "production"  # Set your environment (production, staging, etc.)

before_script:
  # Install Terraform and AWS CLI
```

```
  - curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add -
  - sudo apt-add-repository "deb https://apt.releases.hashicorp.com $(lsb_release -cs) main"
  - sudo apt-get update && sudo apt-get install terraform
  - terraform --version

# Install dependencies and set up AWS credentials
install_dependencies:
  stage: install
  script:
    - echo "Installing dependencies..."
    - apt-get update -y
    - apt-get install -y jq curl
    - echo "$AWS_ACCESS_KEY_ID" | aws configure set aws_access_key_id
    - echo "$AWS_SECRET_ACCESS_KEY" | aws configure set aws_secret_access_key
    - echo "$AWS_DEFAULT_REGION" | aws configure set region
  only:
    - master
  tags:
    - terraform

# Stage: Lint the Terraform configuration
lint_terraform:
  stage: lint
  script:
    - echo "Running terraform fmt (auto-formatting)..."
    - terraform fmt -check -diff
    - echo "Running terraform validate..."
    - terraform validate
  only:
    - master
  tags:
    - terraform

# Stage: Terraform validate the configuration
validate_terraform:
  stage: validate
  script:
    - echo "Validating Terraform configuration..."
    - terraform validate
  only:
    - master
  tags:
    - terraform

# Stage: Plan the Terraform deployment
terraform_plan:
```

PATHNEX
Shaping Careers, Building Futures

```
  stage: plan
  script:
    - echo "Running terraform plan..."
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET" -backend-
config="key=$TF_STATE_KEY"
    - terraform plan -out=tfplan
  artifacts:
    paths:
      - tfplan
    expire_in: 1 hour
  only:
    - master
  tags:
    - terraform

# Stage: Apply the Terraform plan (Apply changes to the cloud)
terraform_apply:
  stage: apply
  script:
    - echo "Applying Terraform configuration..."
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET" -backend-
config="key=$TF_STATE_KEY"
    - terraform apply "tfplan"
  only:
    - master
  when: manual  # This requires manual approval to prevent accidental deployments
  tags:
    - terraform

# Stage: Cleanup any sensitive files
cleanup:
  stage: cleanup
  script:
    - echo "Cleaning up sensitive files..."
    - rm -rf .terraform* terraform.tfstate* tfplan
  only:
    - master
  tags:
    - terraform
```

PATHNEX
Shaping Careers, Building Futures